SALESFORCE EXPERIENCE CLOUD

Customisation tiers for Salesforce websites

Where design work ends, where developer effort begins, and where customisation starts to put Salesforce upgrades at risk.

October 2026 · informed by importing the MSD Figma design system into SLDS 2 tokens, skinning seven standard components in an LWR prototype, and conversations with two Salesforce architects.

Five tiers

Configure, Style, Layout, Component, Override. Each tier adds capability and cost.

Developer gate

Between Style and Layout. Beyond it, changes are code. Expected for client work, and sized by effort.

Upgrade gate

Between Component and Override. Beyond it, each Salesforce release can break the site.

Summary

A Salesforce website can be customised at five levels. Designers prototype designs as Lightning web components in the Lightning Web Runtime (LWR) and hand those prototypes to developers to build in Salesforce.

Client-facing Experience Cloud work is custom by default. Both Salesforce architects we consulted said so: standard components are a starting point, and screens are composed from them with a layer of CSS on top. Tiers 1 and 2 are the base every build stands on, not the goal. The useful questions are how much effort a screen takes, and whether it restructures or reaches into a standard component. Composing standard components with data mapped in is low effort. Restructuring one means building a different component. Reaching into its internals is where upgrade risk sits.

Styling has hard limits. Components honour only some of the hooks Salesforce publishes, and many visual properties have none. Measured in a browser, the skinned button matched 86% of its design's values and the form controls 70%. Component hooks are also a developer preview in SLDS 2. The fidelity gap closes only through a design change or a rebuild, so the **design system must be drawn to what the standard components can do**.

We recommend aligning the design system first, budgeting for composed components from the start, treating rebuilds and the upgrade gate as explicit decisions, and handing over the LWR prototype with its styling register.

The tiers at a glance

	TIER	WHAT CHANGES	WHO	UPGRADE RISK
1	**Configure**	Pages, standard components, navigation, audiences, and the Theme panel's brand colours, fonts and heading styles in Experience Builder	Admin, designer	None
2	**Style**	Values and published names only: the theme as global tokens, per-component styling hooks (a developer preview in SLDS 2), and SLDS utility classes. No new CSS rules	Designer, working from a verified hook inventory	Low: re-verify each release
DEVELOPER GATE	Code, source control, testing and deployment from here down. Expected for client work			
3	**Layout**	Custom theme, page and component layouts built using standard base components, and the CSS classes we write to style them	Developer, designer	Low
4	**Component**	**Compose:** new components assembled from standard ones, with data mapped in. **Rebuild:** a standard component restructured, or rebuilt because no hook reaches its look	Developer	Low, owned by us
UPGRADE GATE	Depends on Salesforce internals that can change without notice			
5	**Override**	Changing how standard components look or behave by reaching into their internal markup or **private styling hooks**	Developer	**High**

Figure 1. Tiers in order of cost. Effort and ongoing maintenance rise with each tier.

Tier 1 · Configure

Everything set in Experience Builder without code: the LWR template, pages, standard components, navigation, audiences, and the Theme panel's colours, fonts and spacing. Theme values are published as `--dxp` styling hooks, which map onto the Lightning design tokens. Anything an admin should change later, such as the brand colour or heading sizes, belongs here rather than in CSS.

Record detail and list pages for an object give the components placed on them the record's context and standard data retrieval. Tier 4 builds on this, so screens designed around them cost less.

Constraints. Only the options Salesforce exposes: the template's theme layouts, the standard section and column components, and each component's Builder properties.

BEFORE TIER 2 · ALIGN THE DESIGN SYSTEM

Tier 2 can only deliver a design drawn in the components' own terms. The Figma library is the contract, so it should:

- **Name every variable after the hook it feeds:** a token the shipped stylesheet declares, with no invented names or aliases.
- **Use SLDS's steps:** colour ramps on 10–95 (plus 45 and 55), and semantic colours pointing where SLDS points, or a recorded reason why not.
- **Bind roles, not hues:** a button fill bound to `accent-container-1`, not to "Electric Blue 50" or a "component hook" variable the component never reads.
- **Draw only what the component has:** its states (no "filled" state), anatomy (no undo button on a plain input) and sizes (16px checkboxes). CSS borders add to a component's size; Figma strokes sit inside it.

Evidence: the first import of the MSD library (347 variables) found 26 code syntaxes naming a wrong or non-existent hook, 14 semantic colours re-pointed away from SLDS, and a font weight one step off. Two library revisions fixed most of these.

Tier 2 · Style

The brand applied by **setting values and naming what Salesforce publishes**. None of the three levers writes a new CSS rule, which keeps this tier on the designer's side of the gate.

Global tokens `--slds-g-*`	The theme: colour ramps and roles, type scale, spacing, radius, shadows. Set once, they reach every standard component. On Experience Cloud they go in the Theme panel where it exposes them, and in site CSS otherwise.
Component hooks `--slds-c-*`	Per-component values, set on the component. Only hooks the component's own stylesheet reads take effect. Where a variant fixes a hook internally, change the global token underneath it on that component instead.
SLDS utility classes `slds-*`	Published classes for spacing, text, grid, sizing and visibility (`slds-m-bottom_small`), specified by name. Salesforce's documented order puts them before hooks: *variant → utility class → styling hook → your own class*. They work only where markup can carry a class: our templates, or content authored in Builder.

Constraints. Standard components encapsulate their markup, so CSS reaches only what their hooks expose. In the components tested, field label size and weight, checkbox and radio size, border width, input padding, focus-ring shape and hover elevation have no hook. A hook can also exist and do nothing (one button hook is broken in Salesforce's own stylesheet), so only the rendered value in a browser proves it.

Production status. Salesforce's SLDS 2 guidance sets three limits. Component hooks are a developer preview, not for production. Assigning values to global hooks is reserved for theming tools. The newest

global hooks are not supported on Experience Cloud sites. The prototype uses component hooks, and re-points globals on single components, to match the design; every such line is listed for the build team in a styling register.

Utilities change between SLDS versions (SLDS 2 removed classes SLDS 1 had) and have gaps, such as responsive spacing and gap. Filling one takes a class of our own, which is Tier 3.

Values the Theme panel does not cover live in a site stylesheet. It holds values, not rules, so it stays at this tier, but it needs one owner, version control and review.

The test for the gate: if a change sets a value or names something Salesforce publishes, it is Tier 2. If it writes a new CSS rule, class or selector, it is Tier 3.

The test for structure: restyling a standard component (colour, spacing, type) stays within Tiers 2 and 3. Changing its structure, meaning what it shows, where and in what order, means building a different component: Tier 4. Reaching into its internals is Tier 5.

THE DEVELOPER GATE

A request needs developer effort when it requires:

- A page structure or region arrangement the template does not offer
- Data the standard components cannot display
- New behaviour: interaction, logic, validation or integration
- A new setting for authors in the Builder panel
- A new CSS rule, class or selector, including a responsive rule SLDS has no utility for
- A look no hook reaches, which the design cannot drop

From here, work is code: source-controlled, tested and deployed. For client-facing screens this is the normal case, so budget for it from the start and size each item by effort. Designers still own the visual specification.

Tier 3 · Layout

Custom theme layouts (header, footer, page frame) and page layouts, built as Lightning web components with regions authors fill in Builder. Usually the first step past the gate, and mostly markup and CSS, so designers and developers work on it closely.

Our own CSS classes live here. A class works only once markup carries it, so it belongs with the layout or component it styles, and we maintain it. Use a project prefix, and never select a Salesforce class or a component's internals: that is Tier 5. A documented `::part` also sits here, as a signed-off last resort: it is published, but styles the part directly rather than through a value.

Constraints. Deployed as metadata, not edited in Builder. Responsiveness and accessibility become ours. Built on public APIs, so upgrade risk is low.

Tier 4 · Component

New Lightning web components with their own markup, styling, data (including Apex), integrations and Builder properties. They come in two kinds, very different in effort:

Compose	Standard components assembled into a new one, with data mapped in: a card repeater for a list of records, or a two-column detail view of badges, fields and text. The standard parts bring their own structure, behaviour and accessibility, so most of the work is layout and data. The normal case, and low effort.
Rebuild	A standard component restructured, or rebuilt because the design needs a look no hook reaches. The supported alternative to Tier 5, but the new control takes on the keyboard behaviour, validation and screen-reader support the standard one provided. Higher effort, so decide it explicitly.

Constraints. Code runs under Salesforce's browser security model, and third-party scripts and domains must be allowed. Performance, accessibility and caching become ours. Upgrade-safe, but ours to maintain.

THE UPGRADE GATE

Salesforce ships three major releases a year. A customisation is at risk when it depends on something Salesforce has not committed to keeping stable. Warning signs:

- A CSS selector targets a class name or element we did not write
- `!important` is used to beat a standard component's styles
- A **private styling hook** is set (`--slds-s-*`, `--_slds-*` or the theme's reference tokens). They look supported but are not; Salesforce's linter flags the first two
- JavaScript modifies a standard component after it renders
- The approach was found by inspecting markup in the browser, not in documentation
- A standard component has been copied and edited instead of rebuilt

Tier 5 · Override

Reaching past a standard component's supported interface: CSS aimed at internal markup, private styling hooks, scripts that alter the page after load, or undocumented behaviour.

Constraints. These often work at first and fail later: a release renames an internal class or retires a private hook, and the live site breaks silently. Salesforce support does not cover them. Each needs regression testing against every release preview.

DECISIONS OPEN

- **Custom icons or the SLDS icon set.** `lightning-icon` renders SLDS's icon library as standard. MSD's own icon suite would be an asset we deliver and maintain. Being checked with the architect.
- **Component hooks in production.** They are a developer preview in SLDS 2, and the prototype relies on them. The build team decides, line by line from the styling register, whether to keep each one, replace it, or accept the component's default.

Recommendations

1. **Align the design system before designing screens.** Keep the Figma library on SLDS's names, steps and roles, and audit every import. Most Tier 2 failures start as a variable that names no real hook.
2. **Design to what ships.** Designers work from a **verified hook inventory**: the hooks each component actually reads and the states they cover, plus the SLDS utilities. Designs it covers are delivered without code.
3. **Review before building, measure after.** Before skinning, map every design value to a lever or to "no lever", and settle the gaps with the designer. After, read the rendered values in a browser.
4. **Decide the fidelity gap explicitly.** For each value no hook reaches: accept the component, change the design, or fund a Tier 4 rebuild. Never drift into Tier 5.
5. **Budget for the developer gate.** Composed Tier 3 and 4 work is expected for client-facing screens: plan it from the start, sized by effort, with an owner for maintenance. Rebuilds are the exception and are approved one by one.
6. **Treat the upgrade gate as an architecture approval.** Tier 5, including any private styling hook, needs sign-off and a documented reason. Prefer a Tier 4 rebuild.
7. **Hand over the prototype and its styling register.** The LWR prototype is built from the same base components a Salesforce build uses, and the architect expects no major translation. The register lists every token and hook set and anything past the upgrade gate; retest it against each release preview.